# EC5207- Devops Engineering

# Assignment

**NANAYAKKARA H.L.U.S.**

**EG/2022/5201**

**22/01/2025**

**CI/CD Design**

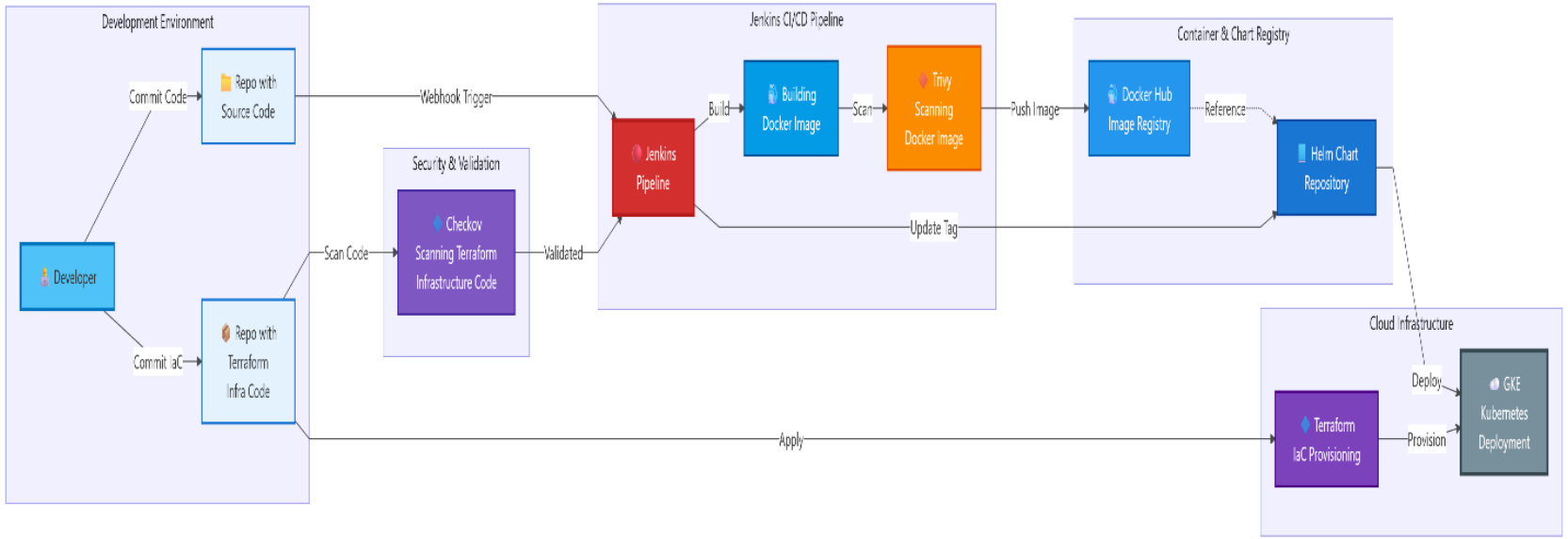
**Code → Build → Scan → Deploy**

1. **Trigger**: Code pushed to GitHub → webhook notifies Jenkins
2. **Build**: Jenkins pipeline (Ubuntu VM) pulls code, builds Docker images for:
    - Node/Express API (backend)
    - React SPA (frontend)
    - MongoDB
3. **Security**: Trivy scans containers → pushes signed images to Docker Hub with versioned tags
4. **Infrastructure**: Terraform provisions GKE clusters, VPC, storage (state in Terraform Cloud)
5. **Deploy**: Helm charts pull tagged images → deploy to Kubernetes
6. **Config**: Ansible manages secrets, configmaps, MongoDB credentials (VM fallback available)

**Runtime Architecture**:

- **Frontend**: NodePort (3000) → public HTTPS Ingress
- **Backend**: ClusterIP (4000) → internal only
- **MongoDB**: StatefulSet with persistent storage
- Network policies restrict DB access to backend pods only

**Tech Stack**: Jenkins 2.440.1, Docker 26.1, Terraform 1.8.2, GKE 1.29.3, Helm 3.14, Ansible 2.16.4, Trivy 0.51


Development Environment
Commit Code
Repo with Source Code
Developer
Commit IaC
Repo with Terraform Infra Code
Webhook Trigger
Scan Code
Security & Validation
Checkov Scanning Terraform Infrastructure Code
Validated
Jenkins CI/CD Pipeline
Jenkins Pipeline
Build
Building Docker Image
Scan
Trivy Scanning Docker Image
Update Tag
Push Image
Container & Chart Registry
Docker Hub Image Registry
Reference
Helm Chart Repository
Cloud Infrastructure
Apply
Terraform IaC Provisioning
Provision
Deploy
GKE Kubernetes Deployment

**Automation Approach**

**Toolchain**:

- **Source & CI/CD**: GitHub → Jenkins 2.440.1 pipelines
- **Containers**: Docker 26.1 (build) → Trivy 0.51 (scan) → Docker Hub (registry)
- **Infrastructure**: Terraform 1.8.2 (GKE provisioning) + Ansible 2.16.4 (config)
- **Deployment**: Helm 3.14 → GKE 1.29.3

**Application Stack**:

- **Backend**: Node.js 18-alpine, Express 4.18, Mongoose 8.18
- **Frontend**: React 19.1, react-router-dom 7.9
- **Database**: MongoDB 7 (containerized)

**Pipeline Flow** (Jenkinsfile):

1. Checkout code
2. Build & tag frontend/backend images
3. Trivy security scan
4. Push to Docker Hub (versioned + latest tags)
5. Rolling update on staging VM (docker-compose.prod)
6. Terraform plan/apply (infra changes)
7. Helm upgrade (deploy new images to GKE)
8. Cleanup temp images

**Key Features**:

- Fully automated: code push → build → scan → deploy (no manual steps)
- PR approvals required (GitHub branch protection)
- Optional npm unit tests stage
- Manual retagging via update-docker-images.sh

**Result**: Zero-touch deployment pipeline for containerized workloads with security scanning and infrastructure-as-code.